

Face Recognition

Name	ID
Ranime Ahmed Elsayed Shehata	21010531
Habiba Tarek Ramadan Ali	21010445
Karene Antoine Nassif Guirguis	21010969

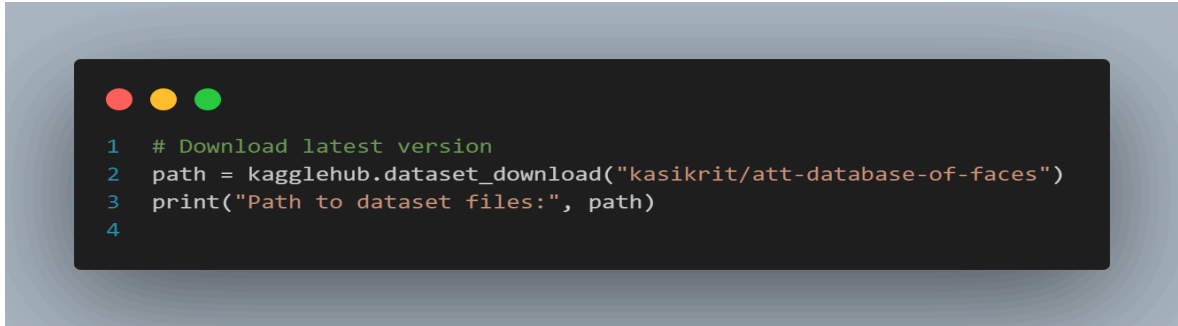
Problem Statement:

We intend to perform face recognition. Face recognition means that for a given image you can tell the subject id. Our database of subjects is very simple. It has 40 subjects. Below we will show the needed steps to achieve the goal of the assignment.

Step 1: Download ORL Dataset:

→ Dataset Description:

- 40 distinct subjects.
- 10 grayscale images per subject (total of 400 images).
- Each image is 112×92 pixels (10,304 features per image).



```
1 # Download latest version
2 path = kagglehub.dataset_download("kasikrit/att-database-of-faces")
3 print("Path to dataset files:", path)
4
```

Step 2: Generating the Data Matrix and the Label vector:

→ Loading the Dataset

- The dataset is downloaded using `kagglehub`.
- Each image is read in grayscale and stored in a list.
- A corresponding label (subject ID from 1 to 40) is stored for each image.

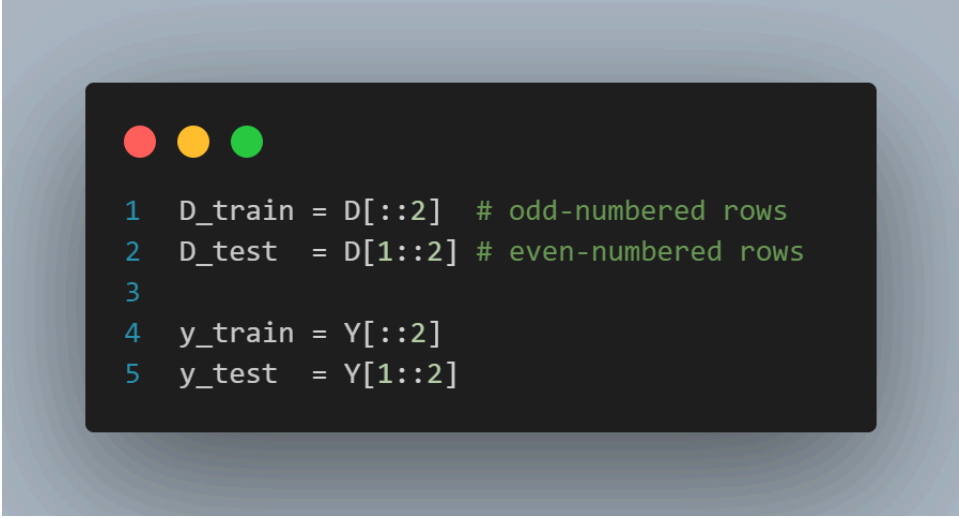
```
images.shape => (400, 112, 92)
Y.shape      => (400, )
```

→ Data Matrix Construction

Images are reshaped to 1D vectors to create a 2D data matrix `D` of shape `(400, 10304)`, where each row is a flattened image.

→ Train-Test Split

- **Training Set:** Odd-indexed samples (200 images)
- **Testing Set:** Even-indexed samples (200 images)



```
1 D_train = D[::2] # odd-numbered rows
2 D_test  = D[1::2] # even-numbered rows
3
4 y_train = Y[::2]
5 y_test  = Y[1::2]
```

Principal Component Analysis (PCA)

PCA is used to reduce the dimensionality of facial images while retaining most of the variance (features).

Steps in PCA Implementation:

- **Mean Centering:** Subtract the mean face from all training images.
- **Covariance Matrix:** Compute covariance matrix of centered data.
- **Eigenvalue Decomposition:** Compute eigenvectors and eigenvalues.
- **Dimensionality Reduction:** Select top k eigenvectors based on explained variance (α).

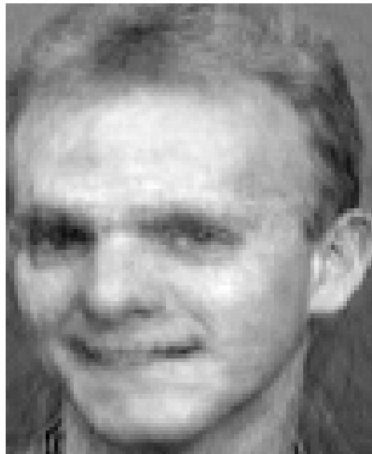
→ Reconstruction Visualization:

Images are reconstructed using different α values (0.5, 0.8, 0.85, 0.9, 0.98).

Higher α retains more variance, leading to better reconstruction.

α	Dimensions	Reconstruction Quality
0.98	155	Near perfect
0.9	76	Clear features
0.85	51	Clear features
0.8	36	Recognizable
0.5	6	Blurry, loss of details

Reconstructed Image with $\alpha=0.98$ and Dimensions =155 Reconstructed Image with $\alpha=0.85$ and Dimensions =51



Reconstructed Image with $\alpha=0.9$ and Dimensions =76 Reconstructed Image with $\alpha=0.8$ and Dimensions =36



Reconstructed Image with $\alpha=0.5$ and Dimensions =6



```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.preprocessing import MinMaxScaler
4 import matplotlib.offsetbox as offsetbox
5
6 class PCA:
7     def __init__(self, D_train, D_test, y_train, y_test):
8         """Class initialization"""
9         self.D_train = D_train
10        self.D_test = D_test
11        self.y_train = y_train
12        self.y_test = y_test
13        self.D_centered = []
14        self.eigenvalues = []
15        self.eigenvectors = []
16
17    def compCov(self):
18        """Compute the covariance matrix"""
19        mean_face = np.mean(self.D_train, axis=0)
20        self.D_centered = self.D_train - mean_face
21        Cov = self.D_centered @ self.D_centered.T
22        Cov = Cov / self.D_train.shape[0]
23        return Cov
24
25    def compEig(self, Cov):
26        """Compute the eigenvalues and the eigenvectors in feature space"""
27        eigenvalues, eigenvectors = np.linalg.eigh(Cov)
28        eigenvalues = eigenvalues[::-1]
29        eigenvectors = eigenvectors[:, ::-1]
30
31        # Convert to feature space
32        eigvecs_feat_space = self.D_centered.T @ eigenvectors
33        eigvecs_feat_space = eigvecs_feat_space[:, :len(eigenvalues)] # reduce to same k
34
35        norm_eigvect = eigvecs_feat_space / np.linalg.norm(eigvecs_feat_space, axis=0)
36
37        self.eigenvalues = eigenvalues
38        self.eigenvectors = norm_eigvect
39
40
41    def selectComp(self, alpha):
42        """Select the first i eigenvectors based on the explained variance"""
43        sum = 0
44        eigsum = np.sum(self.eigenvalues)
45        for i, val in enumerate(self.eigenvalues):
46            sum += val
47            if sum / eigsum >= alpha:
48                return self.eigenvectors[:, :i+1]
49
50    def project(self, alpha):
51        """Project the data on the eigenvectors"""
52        Cov = self.compCov()
53        self.compEig(Cov)
54        eigvec = self.selectComp(alpha)
55        proj = self.D_centered @ eigvec
56        return proj, eigvec
57
58    def visualize_projection(self, alpha, image_shape=(112, 92)):
59        """Image reconstruction"""
60        projected, subeigvec = self.project(alpha)
61        recon = projected[22] @ subeigvec.T
62        mean_face = np.mean(self.D_train, axis=0)
63        recon_image = recon + mean_face
64        recon_image = recon_image.reshape(image_shape)
65
66        plt.imshow(recon_image, cmap='gray')
67        plt.title(f"Reconstructed Image with alpha={alpha} and Dimensions ={projected.shape[1]}")
68        plt.axis('off')
69        plt.show()
70
71
72    pca = PCA(D_train, D_test, y_train, y_test)
73    pca.visualize_projection(alpha=0.98)
74    pca.visualize_projection(alpha=0.85)
75    pca.visualize_projection(alpha=0.9)
76    pca.visualize_projection(alpha=0.8)
77    pca.visualize_projection(alpha=0.5)
78

```

Clustering with K-Means

A custom K-Means class is implemented from scratch, supporting multiple clusters K by minimizing within cluster variance.

Implementation:

- Random initialization of centroids.
- Iteratively update centroids using Lloyd's algorithm.

```
1 class KMeans:
2     def __init__(self, n_clusters, max_iter=100):
3         """Class initialization"""
4         self.n_clusters = n_clusters
5         self.max_iter = max_iter
6
7     def fit(self, X):
8         """Fit the KMeans model to the data"""
9         # Randomly initialize centroids
10        np.random.seed(42)
11        indices = np.random.choice(X.shape[0], self.n_clusters, replace=False) # Randomly select initial centroids
12        self.centroids = X[indices]
13
14        for _ in range(self.max_iter):
15            # Compute distances from each point to each centroid
16            distances = np.linalg.norm(X[:, np.newaxis] - self.centroids, axis=2)
17            self.labels_ = np.argmin(distances, axis=1) + 1 # Shift labels to start from 1
18
19            new_centroids = np.array([
20                X[self.labels_ == i].mean(axis=0) if np.any(self.labels_ == i) else self.centroids[i - 1]
21                for i in range(1, self.n_clusters + 1)
22            ])
23
24            # Check for convergence
25            if np.allclose(self.centroids, new_centroids):
26                break
27
28            self.centroids = new_centroids
29
30            # print("Updated centroids:\n", self.centroids)
31            # print("Updated labels:\n", self.labels_)
32            # print("Updated centroids shape:", self.centroids.shape)
33            # print("Updated labels shape:", self.labels_.shape)
34
35
36    def predict(self, X):
37        """Predict the closest cluster each sample in X belongs to"""
38        distances = np.linalg.norm(X[:, np.newaxis] - self.centroids, axis=2)
39        return np.argmin(distances, axis=1) + 1 # Shift labels to start from 1
40
```

Evaluation Metric (K-Means)

- **Clustering Accuracy:** Computed using the Hungarian algorithm to match predicted clusters with true labels optimally.
- **F1-Score:** Macro-averaged F1-score.
- **Confusion Matrix:** Visualizes true vs. predicted labels.

Experimental Results (K-Means)

K-Means was run on PCA-reduced data with:

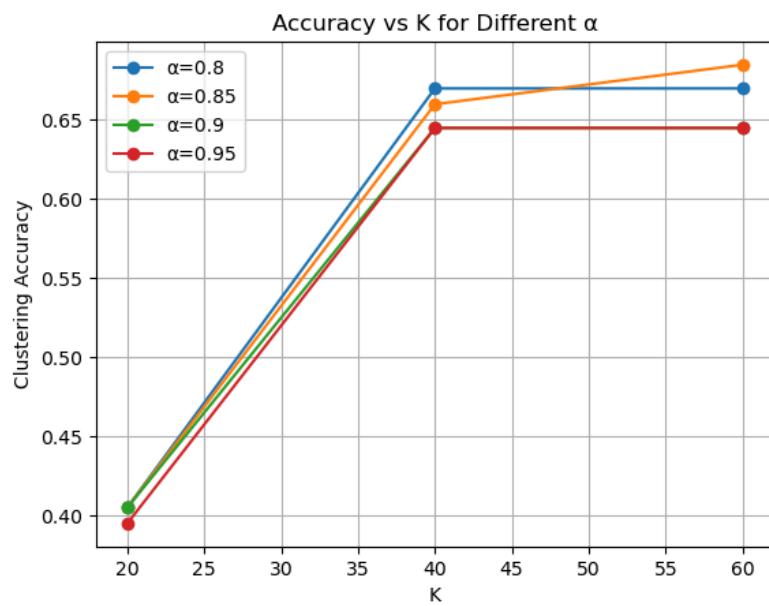
- $\alpha \in \{0.8, 0.85, 0.9, 0.95\}$
- $K \in \{20, 40, 60\}$

Training Accuracy Results (K-Means):

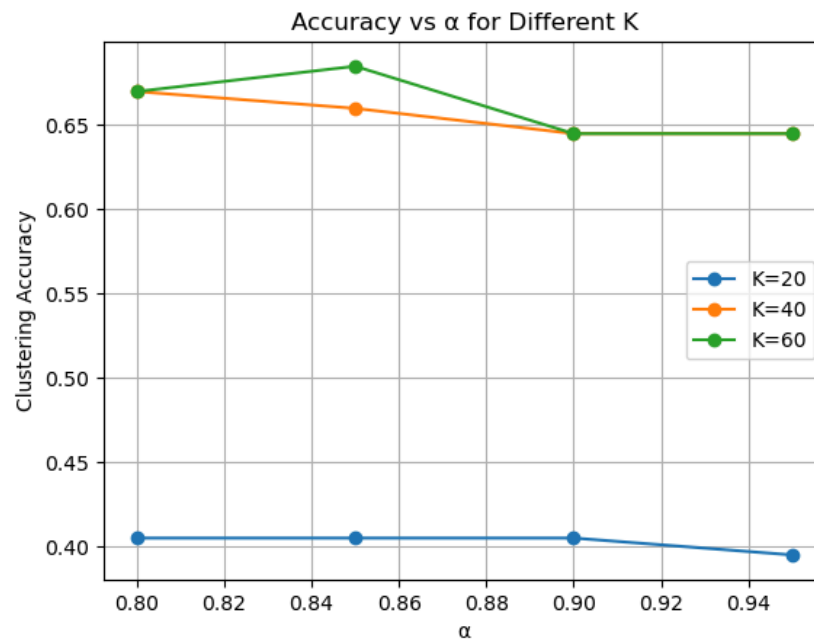
α	K = 20	K = 40	K = 60
0.8	0.405	0.67	0.67
0.85	0.405	0.66	0.685
0.9	0.405	0.645	0.645
0.95	0.395	0.645	0.645

Best Model: $\alpha=0.85$, $K=60$ (68.5% accuracy)

→ Accuracy vs K (for fixed α)



→ Accuracy vs α (for fixed K)



→ Can you find a relation between alpha and classification accuracy?

For fixed K, accuracy is relatively stable across α values (40-68.5%)

Best α : 0.85 (peak accuracy at K=60)

Higher α (≥ 0.9) shows slight degradation, suggesting overfitting to training data and noise amplification in higher dimensions

- As alpha increases, the classification accuracy decreases due to overfitting and it is more sensitive to noises

→ Can you find a relation between K and classification accuracy?

Accuracy improves with K (e.g., 40.5% → 68.5% for $\alpha=0.85$), then plateaus.

- Because more clusters capture finer facial variations, but too many may overfit.
-

Evaluation on Test Set (K-Means)

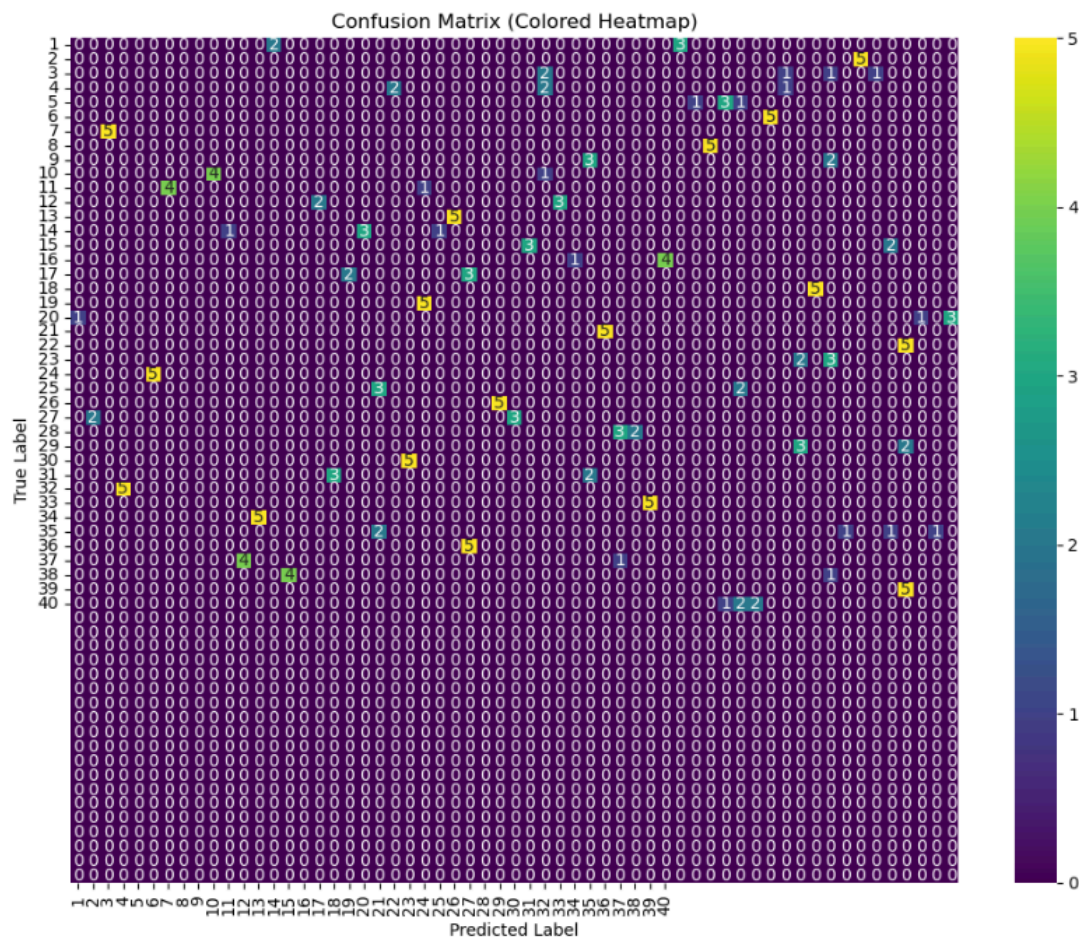
Using the best configuration ($\alpha = 0.85$, $K = 60$):

- Test set was projected using the same PCA eigenvectors.
- KMeans was trained on training projections and used to predict test labels.
- **Evaluation metrics:**

Clustering Accuracy: 0.74 = 74%

Macro F1 Score: 0.188

Confusion Matrix



Clustering with GMM

Implemented the Expectation-Maximization (EM) algorithm from scratch for Gaussian Mixture Models.

Key Methods:

- **fit(X)**: Learns GMM parameters (means, covariances, weights) from data
- **_calculate_responsibilities(X)**: E-step → Computes probabilistic assignments of points to clusters
- **_update_parameters(X, responsibilities)**: M-step → Updates GMM parameters using current assignments
- **predict(X)**: Assigns points to most probable cluster (hard clustering)

```
1 class GMM:
2     def __init__(self, n_components, max_iter=100, tol=1e-6, regularization=1e-6):
3         """Class initialization"""
4         self.n_components = n_components
5         self.max_iter = max_iter
6         self.tol = tol
7         self.regularization = regularization # for regularization
8
9     def fit(self, X):
10        """Fit the GMM model to the data"""
11        np.random.seed(42)
12        n_samples, n_features = X.shape
13
14        # initialize parameters
15        self.means = X[np.random.choice(n_samples, self.n_components, replace=False)]
16        self.covariances = np.array([np.cov(X, rowvar=False)] * self.n_components)
17        self.weights = np.ones(self.n_components) / self.n_components
18
19        for _ in range(self.max_iter):
20            responsibilities = self._calculate_responsibilities(X)
21            self._update_parameters(X, responsibilities)
22
23    def _calculate_responsibilities(self, X):
24        """Calculate the responsibilities (E-step)"""
25        n_samples, n_features = X.shape
26        responsibilities = np.zeros((n_samples, self.n_components))
27
28        for i in range(self.n_components):
29            diff = X - self.means[i]
30            covariance_inv = np.linalg.inv(self.covariances[i] + self.regularization * np.eye(n_features)) # add regularization
31            exponent = -0.5 * np.sum(diff @ covariance_inv * diff, axis=1)
32            responsibilities[:, i] = np.exp(exponent) * self.weights[i]
33        responsibilities /= responsibilities.sum(axis=1, keepdims=True) # normalize
34
35        return responsibilities
36
37    def _update_parameters(self, X, responsibilities):
38        """Update the parameters (M-step)"""
39        n_samples, n_features = X.shape
40        weights = responsibilities.sum(axis=0)
41
42        self.means = responsibilities.T @ X / weights[:, np.newaxis] # update means
43
44        for i in range(self.n_components):
45            diff = X - self.means[i]
46            weighted_diff = responsibilities[:, i][:, np.newaxis] * diff
47            self.covariances[i] = weighted_diff.T @ diff / weights[i] # update covariances
48
49        self.weights = weights / n_samples # update weights
50
51    def predict(self, X):
52        """Predict the closest cluster each sample belongs to"""
53        responsibilities = self._calculate_responsibilities(X)
54        return np.argmax(responsibilities, axis=1)
```

Evaluation Metric

- **Clustering Accuracy:** Computed using the Hungarian algorithm to match predicted clusters with true labels optimally.
- **F1-Score:** Macro-averaged F1-score.
- **Confusion Matrix:** Visualizes true vs. predicted labels.

Experimental Results

GMM was run on PCA-reduced data with:

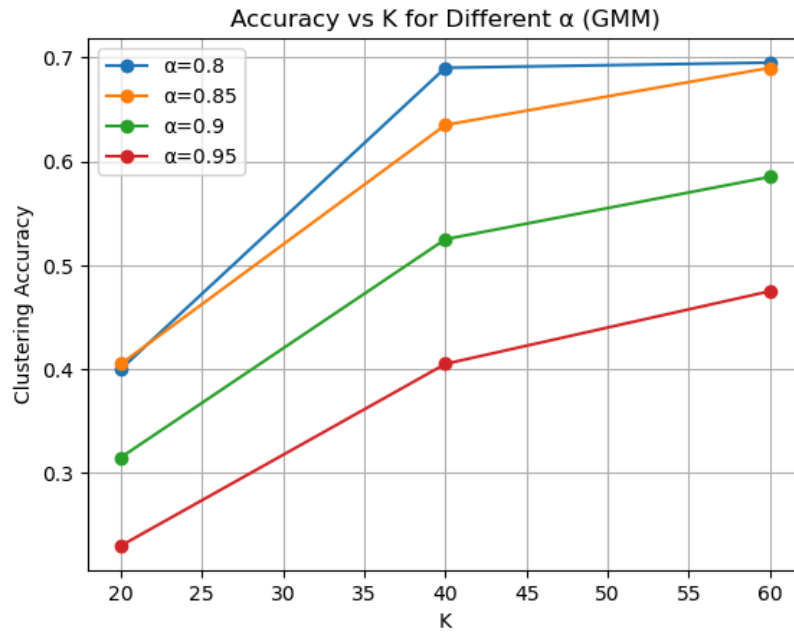
- $\alpha \in \{0.8, 0.85, 0.9, 0.95\}$
- $K \in \{20, 40, 60\}$

Training Accuracy Results:

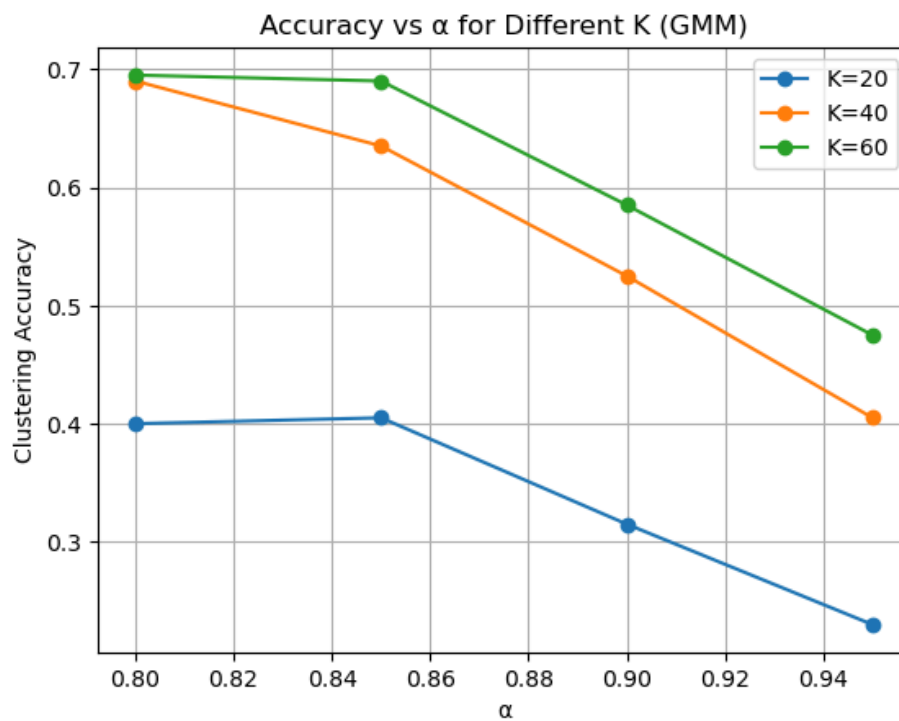
α	K = 20	K = 40	K = 60
0.8	0.4	0.69	0.695
0.85	0.405	0.635	0.69
0.9	0.315	0.525	0.585
0.95	0.23	0.405	0.475

Best Model: $\alpha=0.8$, $K=60$ (69.5% accuracy)

Accuracy vs K (for fixed α)



Accuracy vs α (for fixed K)



→ Can you find a relation between alpha and classification accuracy?

Strong negative correlation between α and accuracy

Best α : 0.80

Higher α reduces accuracy by up to 43% (0.69→0.23 at K=60)

- GMM overfits with high-dimensional data (covariance matrices become unreliable).

→ Can you find a relation between K and classification accuracy?

Trend: Accuracy increases monotonically with K

At $\alpha=0.80$:

K=20 → 40%

K=40 → 69%

K=60 → 69.5%

Optimal K: 60 (but requires low $\alpha=0.8$)

- GMM benefits from more flexible cluster shapes and needs more clusters to model complex distributions but suffers in high dimensions.
-

Evaluation on Test Set

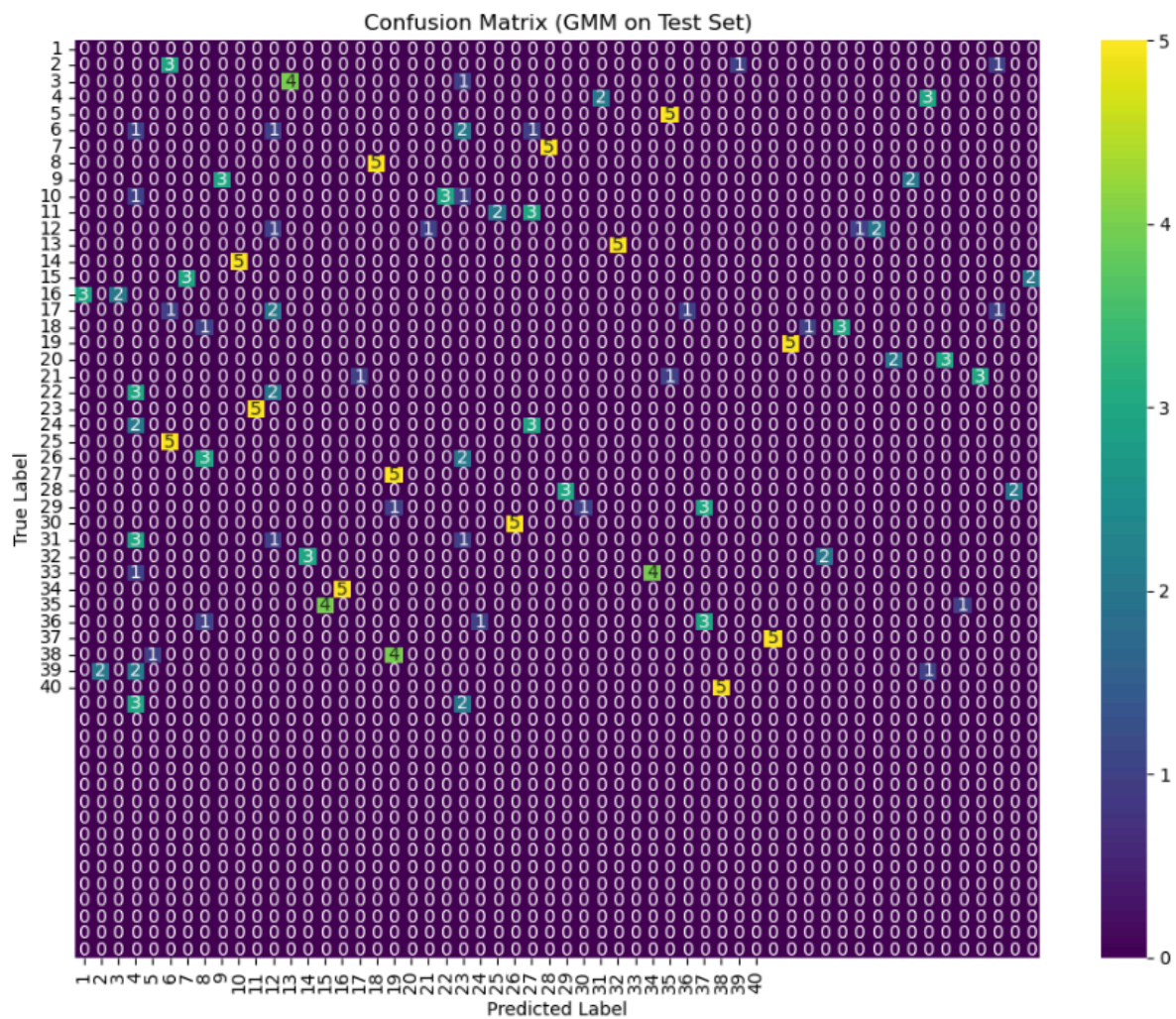
Using the best configuration ($\alpha = 0.8$, $K = 60$):

- Test set was projected using the same PCA eigenvectors.
- Evaluation metrics:

Clustering Accuracy: 0.665 = 66.5%

Macro F1 Score: 0.0164

Confusion Matrix



Compare the performance of GMM to K-Means:

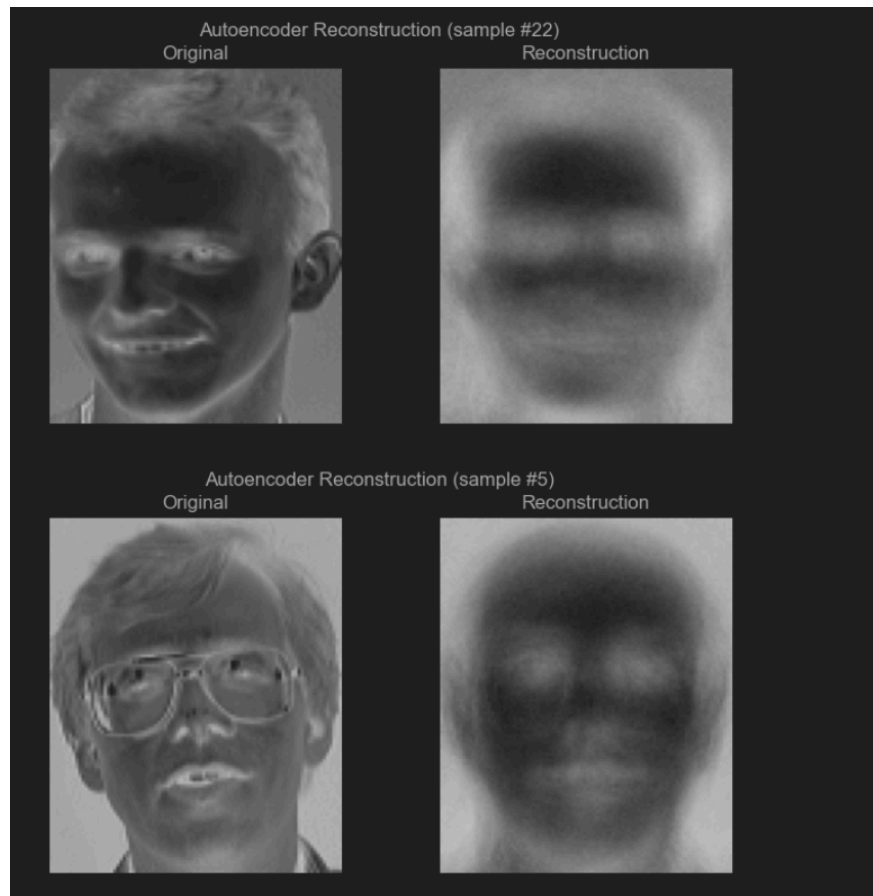
Metrics	K-Means	GMM
Best Accuracy	74.0% ($\alpha=0.85$, $K=60$)	73.5% ($\alpha=0.8$, $K=60$)
F1 Score	0.0188	0.0050
Cluster Shape	Spherical (hard boundaries)	Elliptical (soft boundaries)
Outlier Handling	Poor (sensitive to outliers)	Better (probabilistic assignments)
Model Distribution	Hard assignment: each point belongs to 1 cluster	Soft assignment: probabilistic cluster membership

Bonus (Autoencoders)

- The autoencoder consists of an encoder and decoder implemented from scratch using NumPy.
- Encoder: Reduces the input dimensionality using a single hidden layer with sigmoid activation.
- Decoder: Reconstructs the original input from the compressed representation.
- Weights are initialized using Xavier initialization to ensure stable training.
- Training minimizes Mean Squared Error (MSE) between input and reconstruction using gradient descent.
- Backpropagation is manually implemented for both layers, including weight and bias updates.
- Early stopping is applied to prevent overfitting.
- A visualization function displays the original and reconstructed images side by side.

Reconstruction images screenshots:

With hidden_dim=4000, lr=0.05, epochs=250



Autoencoder Reconstruction (sample #150)
Original Reconstruction



Autoencoder Reconstruction (sample #20)
Original Reconstruction



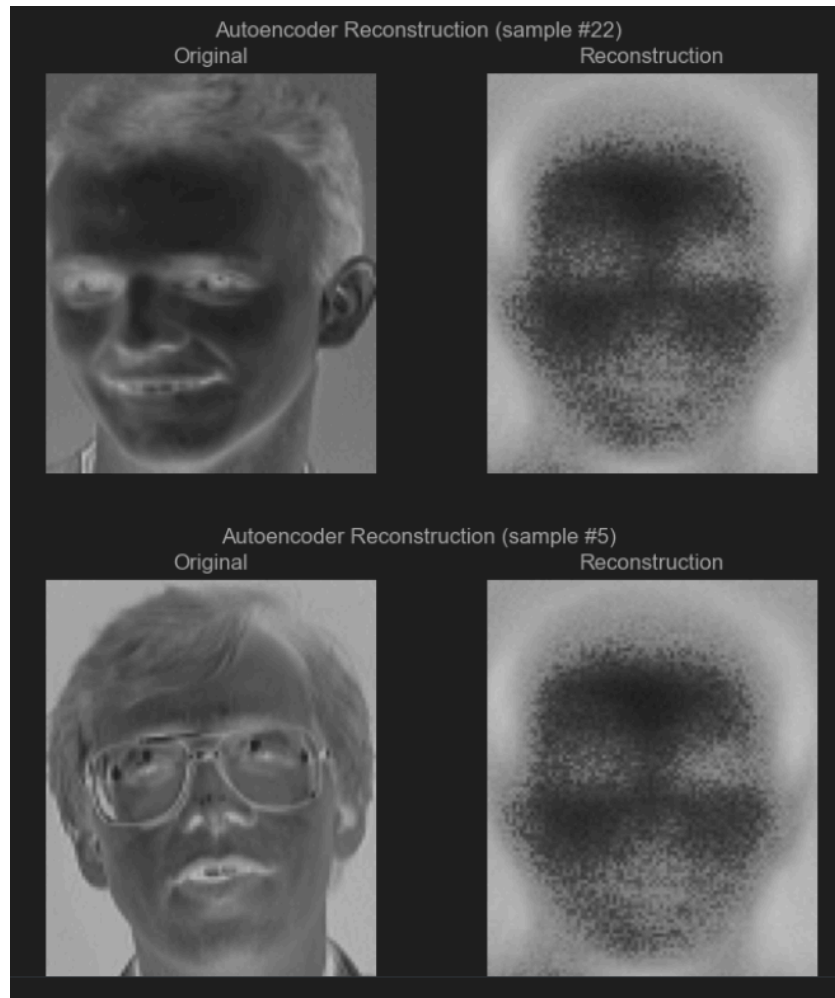
Autoencoder Reconstruction (sample #150)
Original Reconstruction



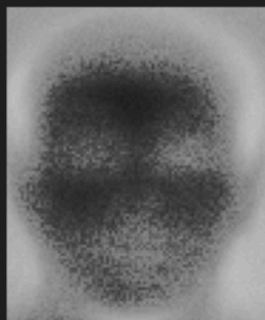
Autoencoder Reconstruction (sample #10)
Original Reconstruction



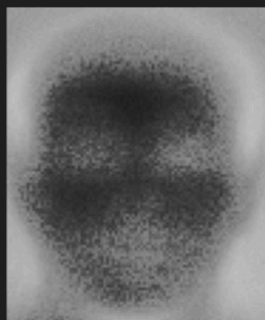
Trying with hidden_dim=500, lr=0.1, epochs=100



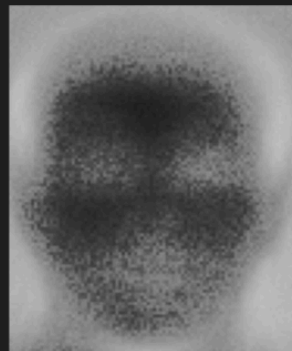
Autoencoder Reconstruction (sample #150)
Original Reconstruction



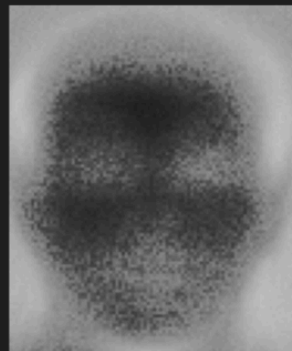
Autoencoder Reconstruction (sample #130)
Original Reconstruction



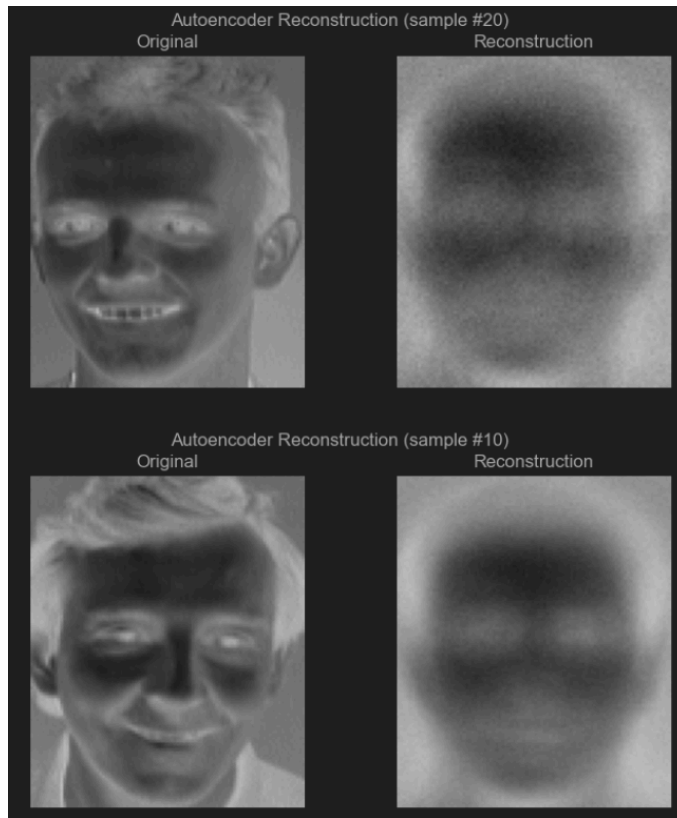
Autoencoder Reconstruction (sample #100)
Original Reconstruction



Autoencoder Reconstruction (sample #110)
Original Reconstruction



Trying with hidden_dim=1000, lr=0.01, epochs=150



Autoencoder Reconstruction (sample #100)
Original Reconstruction



Autoencoder Reconstruction (sample #110)
Original Reconstruction

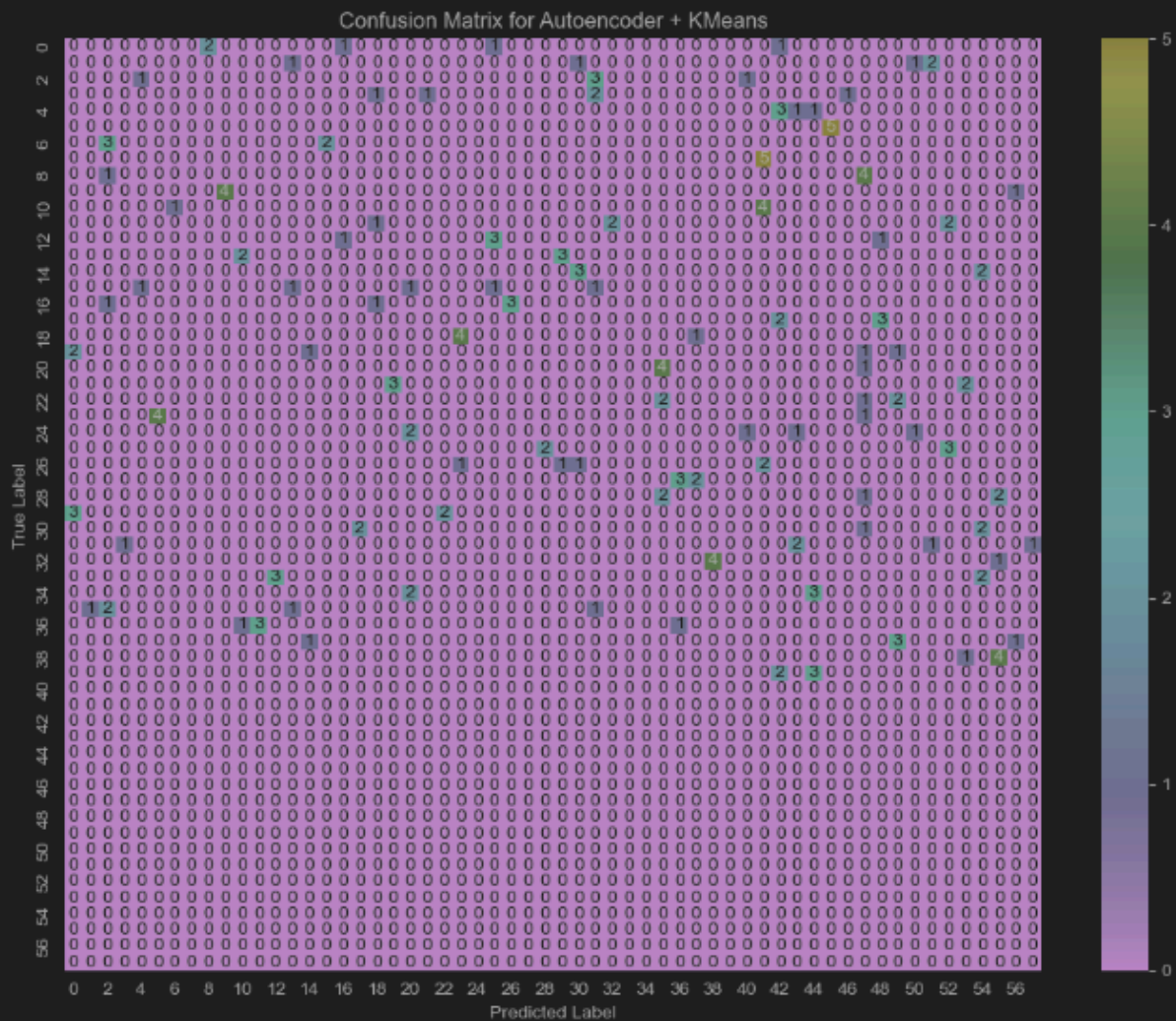


K-Means results with Autoencoder:

Best K: 60, Accuracy: 0.5300, Hidden Dim: 500, Learning Rate: 0.05, Epochs: 250

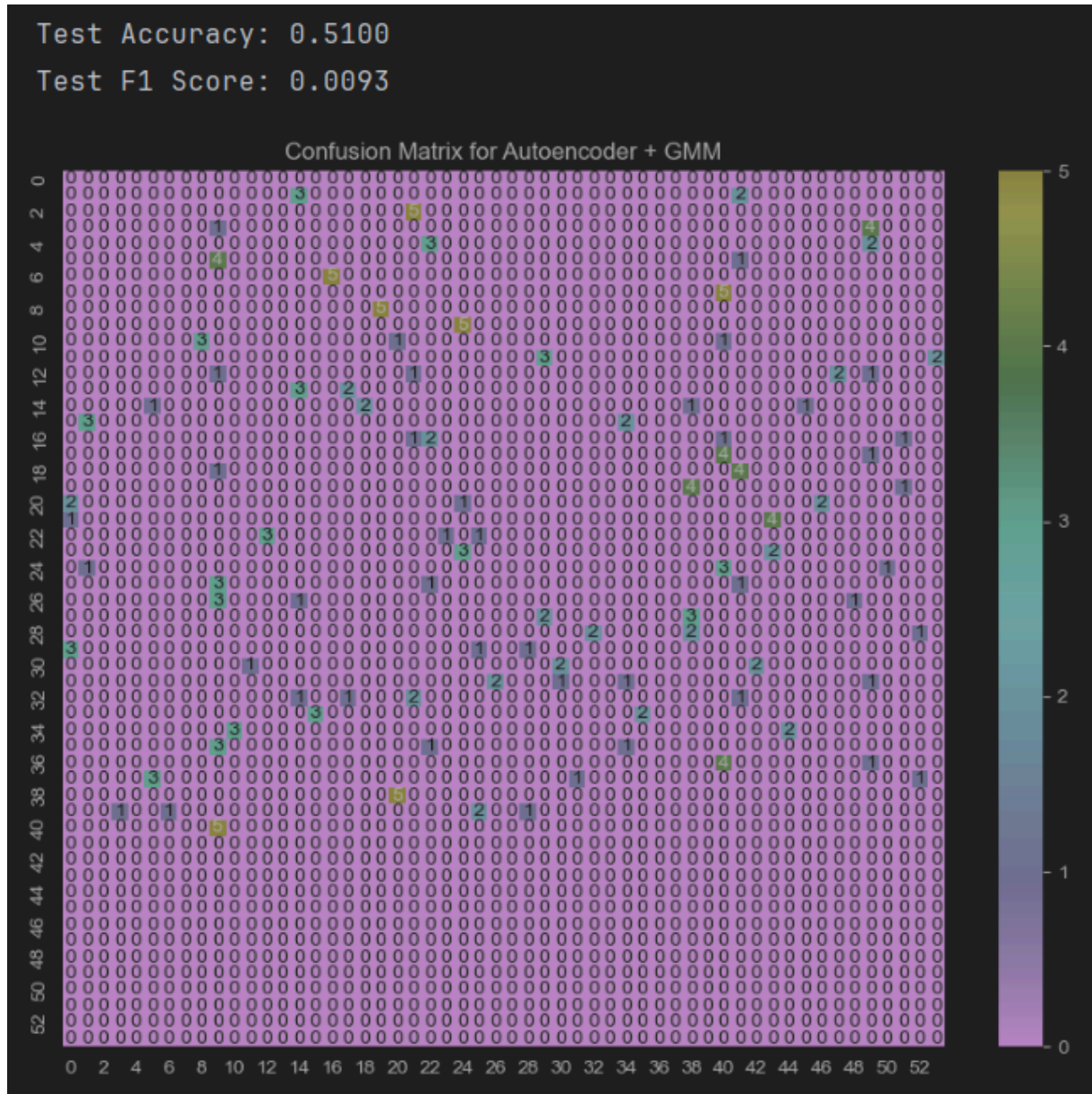
Test Accuracy: 0.5200

Test F1 Score: 0.0192



GMM results with Autoencoder:

```
Best K: 60, Accuracy: 0.5100, Hidden Dim: 250, Learning
Rate: 0.05, Epochs: 60
```



Comparison:

Result of PCA Vs. Results of Autoencoders → PCA performed much better with both, K-Means and GMM, than Autoencoders.

Clustering Technique	PCA	Autoencoders
K-Means	74%	52%
GMM	66.5%	51%
